SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, KATTANKULTHUR

SCHOOL OF COMPUTING

21CSC205P DATABASE MANAGEMENT SYSTEMS LABORATORY WORKBOOK

PROGRAMME:

SPECIALIZATION:

REGISTRATION NUMBER:

STUDENT NAME:

YEAR/SEMESTER/SECTION:

ACADEMIC YEAR: 2025-26 (EVEN)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

SHOOL OF COMPUTING

21CSC205P – DATABASE MANAGEMENT SYSTEMS

RUBRICS FOR EVALUATION

SNo	Date	List of Experiments	Command Execution (3)	Viva (2)	Total (5)	Signature
1		DDL and DML Commands				
2		DQL Commands				
3a		TCL and DCL Commands				
3b		ER Diagram				
4a		Writing relational Algebra and tuple relational calculus, domain relational calculus				
4b		Aggregate Functions, Constraints, Set operations				
5		Sub queries, Joins and Views				
6		PL/SQL- Procedures and Functions, Cursors and Triggers, Exception Handling				
7a		Normalization – 1NF, 2 NF, 3NF				
7b		Normalization - 4NF, 5NF				
8		Transaction and concurrency control				
9		Database Connectivity with Front End Tools.				

Completion date:

Faculty Signature:

Expt No: 1 **DDL and DML Commands**
Date:

Aim: To study DDL and DML commands

DDL Commands:

S.No	Command	Purpose	Syntax	Example	Output
1.	CREATE	Creates a new database	CREATE DATABASE database_name;	CREATE DATABASE College;	Database created.
2.	USE	Selects a database	USE database_name;	USE College;	Database changed.
3.	CREATE	Creates a new table	CREATE TABLE table_name (column datatype);	CREATE TABLE Student (RollNo INT, Name VARCHAR(20), Age INT);	Table created.
4.	SHOW	Displays all tables	SHOW TABLES;	SHOW TABLES;	List of tables displayed.
5.	DESC	Displays table structure	DESC table_name;	DESC Student;	Table structure displayed.
6.	ALTER	Adds a new column	ALTER TABLE table_name ADD column datatype;	ALTER TABLE Student ADD Email VARCHAR(30);	Table altered.
7.	ALTER	Modifies a column	ALTER TABLE table_name MODIFY column datatype;	ALTER TABLE Student MODIFY Name VARCHAR(40);	Table altered.
8.	ALTER	Deletes a column	ALTER TABLE table_name DROP column;	ALTER TABLE Student DROP Age;	Table altered.
9.	RENAME	Renames a table	RENAME TABLE old_name TO new_name;	RENAME TABLE Student TO Student_Details;	Table renamed.
10.	TRUNCATE	Deletes all records	TRUNCATE TABLE table_name;	TRUNCATE TABLE Student_Details;	Table truncated.
11.	DROP	Deletes a table	DROP TABLE table_name;	DROP TABLE Student_Details;	Table dropped.
12.	DROP	Deletes a database	DROP DATABASE database_name;	DROP DATABASE College;	Database dropped.

Practice:

DML Commands

S.No	Command	Purpose	Syntax	Example	Output
1.	INSERT	Adds new records into a table	INSERT INTO table_name VALUES (values);	INSERT INTO Student VALUES (1, 'Arun', 20);	1 row inserted.
2.	INSERT	Inserts selected columns	INSERT INTO table_name (col1, col2) VALUES (values);	INSERT INTO Student (RollNo, Name) VALUES (2, 'Kumar');	1 row inserted.
3.	UPDATE	Modifies existing records	UPDATE table_name SET column=value WHERE condition;	UPDATE Student SET Age = 21 WHERE RollNo = 1;	1 row updated.
4.	DELETE	Removes records	DELETE FROM table_name WHERE condition;	DELETE FROM Student WHERE RollNo = 2;	1 row deleted.

Practice:

Result

Thus, DDL and DML commands were studied successfully.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 2 **DQL Commands**
Date:

Aim: To study DQL commands

DQL Commands:

S.No	Command	Purpose	Syntax	Example	Output
1.	SELECT	Displays all columns	SELECT * FROM table_name;	SELECT * FROM Student;	Displays all data from Student table.
2.	SELECT	Displays selected columns	SELECT column1, column2 FROM table_name;	SELECT Name, Age FROM Student;	Selected data displayed.
3.	SELECT	Filters records based on condition	SELECT * FROM table_name WHERE condition;	SELECT * FROM Student WHERE Age > 20;	Matching records displayed.
Clauses					
1.	LIKE	Searches using pattern	SELECT * FROM table_name WHERE column LIKE pattern;	SELECT * FROM Student WHERE Name LIKE 'A%';	Matching records displayed.
2.	ORDER BY	Sorts the result	SELECT * FROM table_name ORDER BY column;	SELECT * FROM Student ORDER BY Age;	Data sorted.
3.	DISTINCT	Removes duplicate records	SELECT DISTINCT column FROM table_name;	SELECT DISTINCT Age FROM Student;	Unique values displayed.
4.	LIMIT	Restricts number of rows	SELECT * FROM table_name LIMIT n;	SELECT * FROM Student LIMIT 5;	Limited rows displayed.
5.	GROUP BY	Groups records	SELECT column, COUNT(*) FROM table_name GROUP BY column;	SELECT Age, COUNT(*) FROM Student GROUP BY Age;	Grouped results displayed.
6.	HAVING	Filters grouped data	SELECT column, COUNT(*) FROM table_name GROUP BY column HAVING condition;	SELECT Age, COUNT(*) FROM Student GROUP BY Age HAVING COUNT(*) > 1;	Filtered groups displayed.
Arithmetic Operators - Used for calculations.					
1.	+	Addition	SELECT column1 + column2 AS alias_name FROM table_name;	SELECT name, marks + 5 AS bonus_marks FROM students;	5 marks added for all students
2.	–	Subtraction	SELECT column1 - column2 AS alias_name FROM table_name;	SELECT name, marks - 10 AS reduced_marks FROM students;	Reduces 10 marks from each student

3.	*	Multiplication	SELECT column1 * column2 AS alias_name FROM table_name;	SELECT name, marks * 2 AS double_marks FROM students;	Doubles the marks
4.	/	Division	SELECT column1 / column2 AS alias_name FROM table_name;	SELECT name, marks / 2 AS half_marks FROM students;	Divides marks by 2
5.	%	Modulus	SELECT column1 % column2 AS alias_name FROM table_name;	SELECT name, marks % 2 AS remainder FROM students;	Shows 0 for even marks, 1 for odd marks
Logical Operators - Combine multiple conditions.					
1.	AND	Applies multiple conditions	SELECT * FROM table_name WHERE condition1 AND condition2;	SELECT * FROM Student WHERE Age > 18 AND Name = 'Arun';	Filtered records displayed.
2.	OR	Applies alternative conditions	SELECT * FROM table_name WHERE condition1 OR condition2;	SELECT * FROM Student WHERE Age > 20 OR Name = 'Kumar';	Filtered records displayed .
3.	NOT	It **reverses (negates) a condition** in the WHERE clause.	SELECT column_name(s) FROM table_name WHERE NOT condition;	SELECT * FROM students WHERE NOT marks > 70;	Displays students with marks ≤ 70
Relational Operators - Used in WHERE clause.					
1.	=	Equal	SELECT * FROM table-name WHERE column_name = value;	SELECT * FROM students WHERE marks = 60;	Displays students with marks = 60
2.	**!= or <>**	Not equal	SELECT * FROM table-name WHERE column_name != value;	SELECT * FROM students WHERE marks != 60;	Displays students with marks except 60
3.	>	Greater than	SELECT * FROM table-name WHERE column_name > value;	SELECT * FROM students WHERE marks > 60;	Displays students with marks > 60
4.	<	Less than	SELECT * FROM table-name WHERE column_name < value;	SELECT * FROM students WHERE marks < 60;	Displays students with marks <60
5.	>=	Greater than or equal	SELECT * FROM table-name WHERE column_name >= value;	SELECT * FROM students WHERE marks >= 60;	Displays students with marks >=60
6.	<=	Less than or equal	SELECT * FROM table-name WHERE column_name <= value;	SELECT * FROM students WHERE marks <= 60;	Displays students with marks <= 60

Practice:

Result

Thus, DQL commands were executed successfully.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 3a **TCL and DCL Commands**
Date:

Aim: To study TCL and DCL commands

TCL Commands:

S.No	Command	Purpose	Syntax	Example	Output
1.	BEGIN TRANSACTION	Starts a new transaction	BEGIN TRANSACTION;	BEGIN TRANSACTION;	Transaction started.
2.	COMMIT	Saves the transaction permanently	COMMIT;	COMMIT;	Transaction committed.
3.	ROLLBACK	Cancels the transaction	ROLLBACK;	ROLLBACK;	Transaction rolled back.
4.	SAVEPOINT	Sets a transaction point	SAVEPOINT savepoint_name;	SAVEPOINT sp1;	Savepoint created.
5.	ROLLBACK TO	Rolls back to a savepoint	ROLLBACK TO savepoint_name;	ROLLBACK TO sp1;	Rolled back to savepoint.

Practice:

DCL Commands

S.No	Command	Purpose	Syntax	Example	Output
1.	GRANT	Gives permission to a user	GRANT privilege ON object TO user;	GRANT SELECT ON Student TO user1;	Privilege granted.
2.	GRANT	Gives all permissions	GRANT ALL ON object TO user;	GRANT ALL ON Student TO user1;	Privileges granted.
3.	REVOKE	Removes permission from a user	REVOKE privilege ON object FROM user;	REVOKE SELECT ON Student FROM user1;	Privilege revoked.

Practice:

Result

Thus, TCL and DCL commands were studied and executed successfully.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

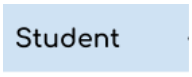
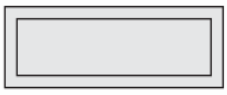
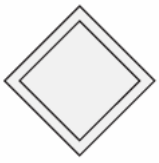
Expt No: 3b **Entity-Relationship Diagram**
Date:

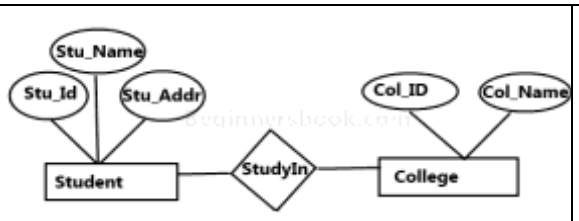
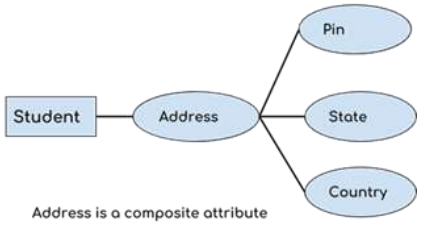
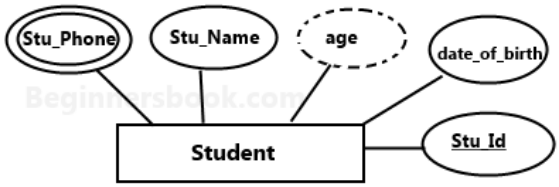
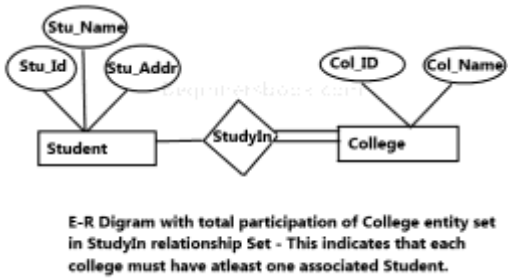
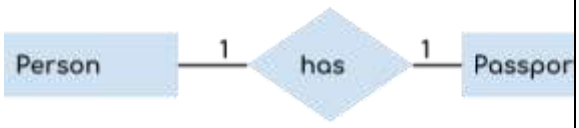
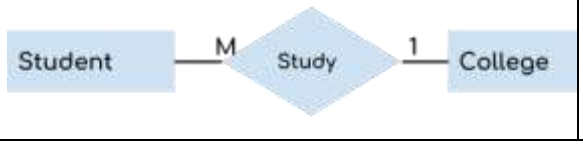
An ER Diagram (Entity–Relationship Diagram) is a visual representation of data and its relationships in a database system. It is used during database design to model the structure of a database before creating tables.

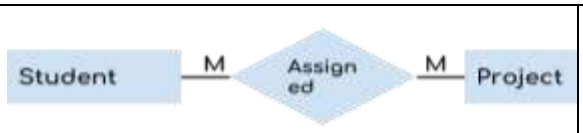
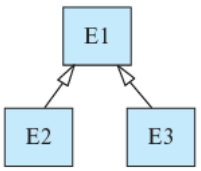
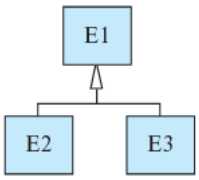
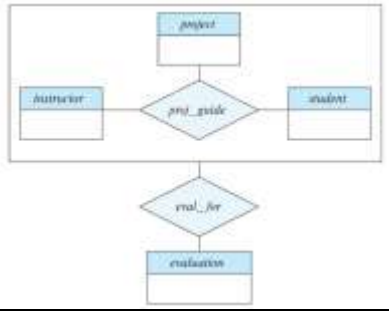
ER diagram has three main components:

1. Entity
    1. Strong Entity
    2. Weak Entity
2. Attribute
    1. Key attribute
    2. Composite attribute
    3. Multivalued attribute
    4. Derived attribute
3. Relationship
    1. One to One
    2. One to Many
    3. Many to One
    4. Many to Many

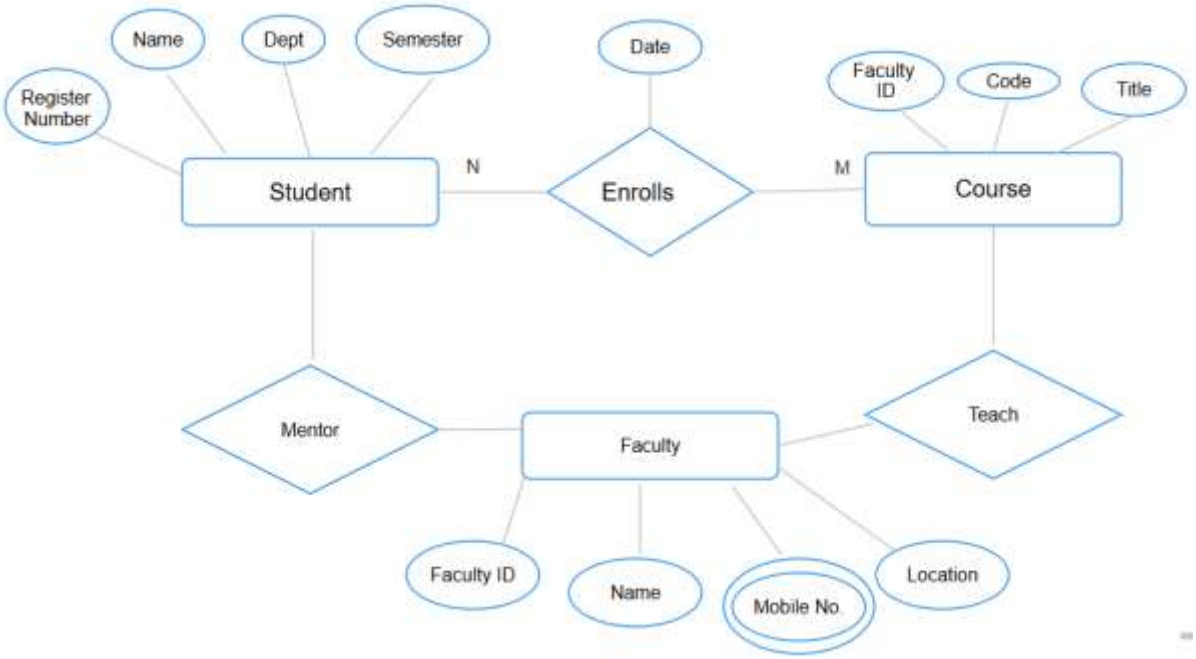
Shapes and their meaning in an E-R Diagram

Shapes	Purpose	Example	Definition
Rectangle	Entity		A real-world object or concept that has an independent existence and can be uniquely identified.
Double Rectangle	Weak Entity		An entity that cannot be uniquely identified by its own attributes and depends on a strong entity.
Ellipses	Attributes		Properties or characteristics that describe an entity or a relationship.
Diamond	Relationship		An association between two or more entities.
Double Diamond	Identifying Relationship		A relationship that connects a weak entity to its owner entity and helps identify it uniquely.

Lines	They link attributes to Entity Sets and Entity sets to Relationship Set		Lines that connect attributes to entity sets and entity sets to relationship sets in an ER diagram.
Ellipses with text underlined	Key Attributes		An attribute that uniquely identifies an entity; represented by underlining.
Ellipses comprising of Ellipses	Composite attribute		An attribute that can be divided into smaller sub-attributes.
Double Ellipses	Multivalued Attributes		An attribute that can have more than one value for a single entity.
Dashed Ellipses	Derived Attributes		An attribute whose value is calculated from other attributes.
Double Rectangles	Weak Entity Sets		A collection of weak entities that depend on a strong entity set for identification.
Double Lines	Total participation of an entity in a relationship set		A constraint where every entity must participate in at least one relationship instance.
Relationship	Cardinality: One to One (1:1)		One entity is related to only one entity in another set.
	Cardinality: One to Many (1:M)		One entity is related to many entities in another set.
	Cardinality: Many to One (M:1)		Many entities are related to one entity in another set.

	Cardinality: Many to Many (M:N)		Many entities are related to many entities in another set.
Extended Features	Overlapping Specialization and Generalization		Overlapping specialization and generalization is a constraint in ER modeling where an entity from a superclass can belong to more than one subclass at the same time.
	Disjoint Specialization and Generalization		Disjoint specialization and generalization is a constraint in ER modeling where an entity from a superclass can belong to only one subclass at a time.
	Aggregation		A higher-level abstraction where a relationship set is treated as an entity set.

Example:



Practice:

Result

Thus, ER diagram was studied and designed successfully.

Command Execution (3)	Viva (2)	Total (5)	Faculty Comments and Signature

Expt No: 4a **Writing relational Algebra and tuple calculus**
Date:

Aim: To Write relational Algebra and tuple calculus

Basic Relational Algebra Operators:

STUDENT(Sid, Sname, Dept, Age)
COURSE(Cid, Cname, Credits, Dept)
ENROLLMENT(Sid, Cid, Grade)

Command	Description	Syntax	Example	Result (SQL Meaning)
Selection (σ)	Selects rows based on condition	σcondition(R)	σDept='CSE'(STUDENT)	SELECT * FROM STUDENT WHERE Dept='CSE';
Projection (π)	Selects columns, removes duplicates	πattributes(R)	πName,Dept(STUDENT)	SELECT DISTINCT Name, Dept FROM STUDENT;
Rename (ρ)	Renames relation or attributes	ρ(NewName, attributes)(R)	ρ(S(SID,Name,Dept,Age))(STUDENT)	Alias in SQL: STUDENT AS S

Set Operations

Command	Description	Syntax	Example	Result (SQL Meaning)
Union (∪)	Combines tuples from both relations	R ∪ S	πSID(CSE) ∪ πSID(ECE)	SELECT SID FROM CSE UNION SELECT SID FROM ECE;
Intersection (∩)	Common tuples	R ∩ S	πSID(CSE) ∩ πSID(CSE_Cloud)	SELECT * FROM R INTERSECT SELECT * FROM S;
Set Difference (−)	Tuples in R not in S	R − S	STUDENT − σ Dept='CSE'(STUDENT)	SELECT * FROM STUDENT EXCEPT SELECT * FROM STUDENT WHERE DEPT=”CSE”;

Cartesian and Join Operations

Command	Description	Syntax	Example	Result (SQL Meaning)
Cartesian Product (×)	All combinations of tuples	R × S	STUDENT × COURSE	SELECT * FROM STUDENT, COURSE;
Theta Join (⋈θ)	Join with condition	R ⋈θ S	STUDENT ⋈ STUDENT.Sid=ENROLLMENT.Sid ENROLLMENT	SELECT * FROM STUDENT JOIN ENROLL ON STUDENT.SID=ENROLL.SID;
Equi Join	Join using equality	R ⋈A=B S	STUDENT ⋈ Sid=Sid ENROLLMENT	JOIN using equality
Natural Join (⋈)	Join on common attributes	R ⋈ S	STUDENT ⋈ ENROLL	NATURAL JOIN

Division & Aggregation (Advanced)

Command	Description	Syntax	Example	Result (SQL Meaning)
Division (÷)	Finds tuples related to ALL in another relation	R ÷ S	ENROLLMENT ÷ π Cid(σ Dept='CSE'(COURSE))	Students enrolled in all courses
Grouping (γ)	Aggregation with grouping	γ group, func(attr)(R)	γ Dept, COUNT(Sid)(STUDENT)	SELECT Dept, COUNT(SID) FROM STUDENT GROUP BY Dept;

Practice:

https://dbis-uibk.github.io/relax/landing (Online tool to execute relational algebra and tuple calculus)

Tuple Relational Calculus

Tuple Relational Calculus – Operator & Concept Table

S.No	Concept	Notation	Purpose	General Form	Example
1	Tuple Variable	t	Represents a tuple	Relation(t)	STUDENT(t)
2	Selection Condition	.attribute	Access tuple attribute	t.attribute	t.Dept = 'CSE'
3	Logical AND	∧	Combines conditions	cond1 ∧ cond2	STUDENT(t) ∧ t.Age > 20
4	Logical OR	∨	Either condition true	cond1 ∨ cond2	t.Dept='CSE' ∨ t.Dept='ECE'
5	Logical NOT	¬	Negates condition	¬ condition	¬(t.Age < 18)
6	Existential Quantifier	∃	Checks existence	∃ t (condition)	∃ e (ENROLLMENT(e))
7	Universal Quantifier	∀	Checks for all tuples	∀ t (condition)	∀ c (COURSE(c))
8	Implication	→	Logical implication	cond1 → cond2	t.Dept='CSE' → t.Age>18

Practice:

Result

Thus, Relational Algebra and Tuple Relational Calculus commands were studied successfully.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 4b **Aggregate Functions, Constraints & Set Operations**
Date:

Aim: To execute Simple Queries, Constraints & Set Operations

Aggregate Functions:

An aggregate function is a function in DBMS that performs a calculation on a set of values (multiple rows) and returns a single summarized result.

Aggregate Function	Description	Syntax (MySQL)	Example Query	Result
COUNT()	Returns the number of rows that match a condition	COUNT(column_name)	SELECT COUNT(*) FROM Student;	Total number of records in Student table
SUM()	Returns the total sum of a numeric column	SUM(column_name)	SELECT SUM(Marks) FROM Student;	Total marks of all students
AVG()	Returns the average value of a numeric column	AVG(column_name)	SELECT AVG(Marks) FROM Student;	Average marks of students
MIN()	Returns the smallest value in a column	MIN(column_name)	SELECT MIN(Marks) FROM Student;	Minimum marks scored
MAX()	Returns the largest value in a column	MAX(column_name)	SELECT MAX(Marks) FROM Student;	Highest marks scored
VARIANCE()	Returns the population variance of a column	VARIANCE(column_name)	SELECT VARIANCE(Marks) FROM Student;	Variance of marks
STDDEV()	Returns the population standard deviation	STDDEV(column_name)	SELECT STDDEV(Marks) FROM Student;	Standard deviation of marks

Practice:

Constraints:

Constraints are rules applied on database columns or tables to restrict the type of data that can be stored, ensuring data accuracy, consistency, and integrity.

Constraint	Description	Syntax (MySQL)	Example	Result
PRIMARY KEY	Uniquely identifies a record	PRIMARY KEY(col)	student_id INT PRIMARY KEY	No duplicate IDs
NOT NULL	Prevents NULL values	col datatype NOT NULL	name VARCHAR(20) NOT NULL	Name mandatory
UNIQUE	Ensures unique values	UNIQUE(col)	email VARCHAR(50) UNIQUE	No duplicate email
CHECK	Restricts values	CHECK(condition)	marks INT CHECK(marks>=0)	Marks ≥ 0
DEFAULT	Assigns default value	DEFAULT value	dept VARCHAR(10) DEFAULT 'CSE'	Default dept set
FOREIGN KEY	References another table	FOREIGN KEY(col) REFERENCES table(col)	dept_id REFERENCES DEPT(id)	Enforces referential integrity

Practice:

Set Operations:

Set operations are SQL operations used to combine the results of two or more SELECT queries into a single result set, following set theory rules.

Operation	Description	Syntax (MySQL)	Example	Result
UNION	Combines results (no duplicates)	query1 UNION query2	SELECT name FROM CSE UNION SELECT name FROM ECE;	Combined unique names
UNION ALL	Combines results (with duplicates)	query1 UNION ALL query2	SELECT name FROM CSE UNION ALL SELECT name FROM ECE;	All names
INTERSECT*	Common records	MySQL workaround	SELECT name FROM CSE WHERE name IN (SELECT name FROM ECE);	Common students
MINUS*	Records in first not in second	MySQL workaround	SELECT name FROM CSE WHERE name NOT IN (SELECT name FROM ECE);	Only CSE students

Practice:

Result

Thus, queries with Aggregate functions, Constraints & Set Operations were studied and executed successfully.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 5 **Subqueries, Joins & Views**
Date:

Aim: To execute Subqueries, Joins & Views

Subqueries:

.No	Operator	Purpose	General Syntax	Example
Single-row Subquery				
1	=	Compares with single value returned by subquery	column = (SELECT expr FROM table)	SELECT * FROM STUDENT WHERE Age = (SELECT MIN(Age) FROM STUDENT);
2	>	Checks greater than subquery result	column > (SELECT expr FROM table)	SELECT * FROM STUDENT WHERE Age > (SELECT AVG(Age) FROM STUDENT)
3	<	Checks less than subquery result	column < (SELECT expr FROM table)	SELECT * FROM STUDENT WHERE Credits < (SELECT MAX(Credits) FROM COURSE)
4	>=	Greater than or equal to subquery value	column >= (SELECT expr FROM table)	SELECT * FROM STUDENT WHERE Age >= (SELECT AVG(Age) FROM STUDENT)
5	<=	Less than or equal to subquery value	column <= (SELECT expr FROM table)	SELECT * FROM STUDENT WHERE Credits <= (SELECT AVG(Credits) FROM COURSE)
Multi-row Subquery				
6	IN	Matches any value from subquery result	column IN (SELECT column FROM table)	SELECT * FROM STUDENT WHERE Sid IN (SELECT Sid FROM ENROLLMENT)
7	NOT IN	Excludes values returned by subquery	column NOT IN (SELECT column FROM table)	SELECT * FROM STUDENT WHERE Sid NOT IN (SELECT Sid FROM ENROLLMENT)
8	ANY	Compares with any one value in subquery	column op ANY (SELECT column FROM table)	SELECT * FROM STUDENT WHERE Age > ANY (SELECT

				Age FROM STUDENT WHERE Dept='CSE')
9	ALL	Compares with all values in subquery	column op ALL (SELECT column FROM table)	SELECT * FROM STUDENT WHERE Age > ALL (SELECT Age FROM STUDENT WHERE Dept='ECE')
Multi-row Subquery				
10	EXISTS	Checks whether subquery returns rows	EXISTS (SELECT * FROM table WHERE condition)	SELECT * FROM STUDENT WHERE EXISTS (SELECT * FROM ENROLLMENT E WHERE E.Sid=STUDENT.Sid)
11	NOT EXISTS	Checks whether subquery returns no rows	NOT EXISTS (SELECT * FROM table WHERE condition)	SELECT * FROM STUDENT WHERE NOT EXISTS (SELECT * FROM ENROLLMENT E WHERE E.Sid=STUDENT.Sid)
12	Nested subquery	Subquery inside another	SELECT column_list FROM table_name WHERE column operator (SELECT column FROM table WHERE condition operator (SELECT column FROM table WHERE condition));	SELECT Dept, AVG(Age) FROM (SELECT Dept, Age FROM STUDENT) T GROUP BY Dept;

Practice:

Joins

Join Type	Description	Syntax (MySQL)	Example Query	Result
INNER JOIN	Matching rows	SELECT ... FROM A INNER JOIN B ON condition;	SELECT name, dept_name FROM STUDENT S INNER JOIN DEPARTMENT D ON S.dept_id=D.dept_id;	Matched records
LEFT JOIN	All left + matched right	LEFT JOIN	SELECT name, dept_name FROM STUDENT S LEFT JOIN DEPARTMENT D ON S.dept_id=D.dept_id;	Includes NULL
RIGHT JOIN	All right + matched left	RIGHT JOIN	SELECT name, dept_name FROM STUDENT S RIGHT JOIN DEPARTMENT D ON S.dept_id=D.dept_id;	All depts
CROSS JOIN	Cartesian product	CROSS JOIN	SELECT name, dept_name FROM STUDENT CROSS JOIN DEPARTMENT;	All combinations
SELF JOIN	Table joined with itself	Alias usage	SELECT A.name, B.name FROM STUDENT A, STUDENT B WHERE A.dept_id=B.dept_id AND A.student_id<>B.student_id;	Same dept students

Practice:

Views

Command	Description	Syntax	Example	Result
CREATE VIEW	Create virtual table	CREATE VIEW view_name AS SELECT...	CREATE VIEW CSE_VIEW AS SELECT name FROM STUDENT WHERE dept_id=1;	View created
SELECT VIEW	Retrieve view data	SELECT * FROM view;	SELECT * FROM CSE_VIEW;	Displays view data
UPDATE VIEW	Modify via view	UPDATE view SET col=value;	UPDATE CSE_VIEW SET name='Raj';	Underlying table updated
DROP VIEW	Remove view	DROP VIEW view_name;	DROP VIEW CSE_VIEW;	View deleted

Practice:

Result

Thus, Subqueries, Joins & Views were studied and executed successfully.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 6 PL/SQL- Procedures and Functions, Cursors and Triggers, Exception Handling
Date:

Aim: The aim is to introduce, explain, and provide hands-on practice on Procedures, Functions Cursors, Triggers, and Exception Handling in PL/SQL.

PL/SQL Procedure

Syntax - Create the Procedure

```
DELIMITER $$
CREATE PROCEDURE procedure_name ()
BEGIN
  statements;
END$$
DELIMITER ;
```

Example

Example 1	**Example 2**	**Example 3**
DELIMITER $$ CREATE PROCEDURE greet() BEGIN SELECT 'Hello World'; END$$ DELIMITER ;	DELIMITER $$ CREATE PROCEDURE insert_employee (IN p_emp_id INT, IN p_emp_name VARCHAR(50), IN p_salary INT) BEGIN INSERT INTO employee (emp_id, emp_name, salary) VALUES (p_emp_id, p_emp_name, p_salary); END$$ DELIMITER ;	DELIMITER $$ CREATE PROCEDURE update_salary (IN p_emp_id INT, IN p_new_salary INT) BEGIN UPDATE employee SET salary = p_new_salary WHERE emp_id = p_emp_id; END$$ DELIMITER ;
Call the Procedure		
Mysql> call greet()	Mysql> CALL insert_employee(101, 'Ravi', 40000);	CALL update_salary(101, 45000);
Output		
Hello World	Query OK, 1 row affected (0.07 sec)	Query OK, 1 row affected (0.03 sec)

```
mysql> select * from employee;
+--------+----------+--------+
| emp_id | emp_name | salary |
+--------+----------+--------+
|    101 | Ravi     |  45000 |
+--------+----------+--------+
```

Practice:

PL/SQL Functions

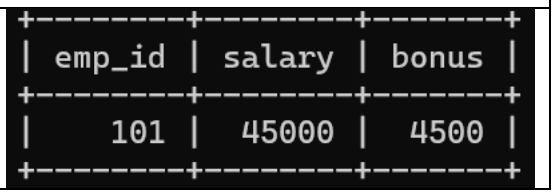
Syntax - Create the Function

```
DELIMITER $$
CREATE FUNCTION function_name (parameter datatype)
RETURNS datatype
DETERMINISTIC
BEGIN
  RETURN value;
END$$
DELIMITER ;
```

Example

Example 1	**Example 2**	**Example 3**

DELIMITER $$
 	
CREATE FUNCTION square_no (n INT)
RETURNS INT
DETERMINISTIC
BEGIN
RETURN n * n;
END$$
 	
DELIMITER ;
	DELIMITER $$
	
	CREATE FUNCTION check_even_odd(n INT)
	RETURNS VARCHAR(10)
	DETERMINISTIC
	BEGIN
	IF n % 2 = 0 THEN
	RETURN 'Even';
	ELSE
	RETURN 'Odd';
	END IF;
	END$$
	
	DELIMITER ;
		DELIMITER $$
		
		CREATE FUNCTION calculate_bonus(sal INT)
		RETURNS INT
		DETERMINISTIC
		BEGIN
		RETURN sal * 0.10;
		END$$
		
		DELIMITER ;

Call the Procedure		
Mysql>SELECT square_no(5);	Mysql> SELECT check_even_odd(15);	Mysql> SELECT emp_id, salary, calculate_bonus(salary) AS bonus FROM employee;
Output		
25	Odd	

Practice:

Cursor:

Concept Explanation:

- A cursor is a database object used to retrieve and process query results one row at a time.
- Cursors are required when a SQL query returns multiple rows and each row needs to be processed individually.
- Cursors are used only inside stored procedures.

Types of Cursors:

1. Implicit Cursor
   - Created automatically by Oracle
   - Used for INSERT, UPDATE, DELETE statements
2. Explicit Cursor
   - Declared and controlled by the programmer
   - Used for SELECT statements returning multiple rows

Cursor Life Cycle:

1. Declare
2. Open
3. Fetch
4. Close

Syntax:

```
DECLARE cursor_name CURSOR FOR SELECT_statement;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
OPEN cursor_name;
read_loop: LOOP
  FETCH cursor_name INTO variables;
  IF done = 1 THEN
    LEAVE read_loop;
  END IF;
  -- process row
END LOOP;
CLOSE cursor_name;
```

Example:

Consider the table EMPLOYEE with the following structure:

Column Name	Datatype
emp_id	int
emp_name	VARCHAR
salary	int
deptid	int

Question:
Write a PL/SQL program using an explicit cursor to display the employee name and salary of all employees from the EMPLOYEE table.

```
DELIMITER $$

CREATE PROCEDURE show_employee_names()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_name VARCHAR(50);

  DECLARE emp_cursor CURSOR FOR
    SELECT emp_name FROM employee;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN emp_cursor;

  read_loop: LOOP
    FETCH emp_cursor INTO v_name;
    IF done = 1 THEN
      LEAVE read_loop;
    END IF;
    SELECT v_name AS Employee_Name;
  END LOOP;

  CLOSE emp_cursor;
END$$

DELIMITER ;
```

Execute the cursor:

```
CALL show_employee_names();
```

Output:

```
+---------------+
| Employee_Name |
+---------------+
| Ravi          |
+---------------+
```

Practice:

Trigger:

Concept Explanation:
A trigger is a stored program that automatically executes when a specific database event occurs on a table.

- Common Trigger Events:
  - INSERT
  - UPDATE
  - DELETE
- Trigger Types:
  - BEFORE Trigger – Executes before the operation
  - AFTER Trigger – Executes after the operation
  - ROW-level Trigger – Executes for each affected row (default)
- **MySQL allows only one event per trigger**

Create Trigger- Syntax

```
DELIMITER $$

CREATE TRIGGER trigger_name

BEFORE | AFTER INSERT | UPDATE | DELETE

ON table_name

FOR EACH ROW

BEGIN

  trigger_body;

END$$

DELIMITER ;
```

Example:

Example 1: BEFORE INSERT Trigger

```
+----------+-------------+------+
| Field    | Type        | Null |
+----------+-------------+------+
| emp_id   | int         | NO   |
| emp_name | varchar(50) | YES  |
| salary   | int         | YES  |
+----------+-------------+------+
```

```
DELIMITER $$

CREATE TRIGGER before_insert_employee
BEFORE INSERT ON employee
FOR EACH ROW
```

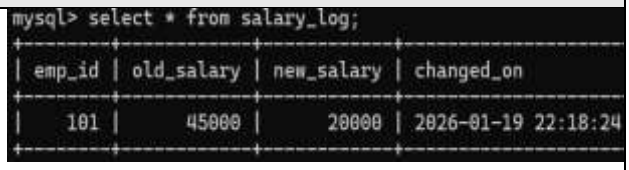
Example 2: AFTER UPDATE Trigger (Audit Log)

```
+-----------+-----------+------+
| Field     | Type      | Null |
+-----------+-----------+------+
| emp_id    | int       | YES  |
| old_salary| int       | YES  |
| new_salary| int       | YES  |
| changed_on| timestamp | YES  |
+-----------+-----------+------+
```

```
DELIMITER $$

CREATE TRIGGER after_update_salary
AFTER UPDATE ON employee
FOR EACH ROW
```

BEGIN
  IF NEW.salary < 10000 THEN
  SET NEW.salary = 10000;
  END IF;
END$$
 	
DELIMITER ;
	BEGIN
	  INSERT INTO salary_log
	  VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
	END$$
	
	DELIMITER ;

Execute Trigger - automatic	
INSERT INTO employee VALUES (102, 'Ram', 8000);	**UPDATE employee SET salary = 20000 WHERE emp_id = 101;**
Output	
	

Practice:

EXCEPTION HANDLING

Concept Explanation

Exception handling allows a program to gracefully handle runtime errors instead of abruptly terminating execution.

Syntax:

```
DECLARE handler_type HANDLER

FOR condition_value

handler_statement;
```

Handler Type:

Handler Type	Purpose
CONTINUE	Continue execution after handling error
EXIT	Exit the block after handling error

Condition Values

Condition	Description
NOT FOUND	Cursor fetch reaches end
SQLEXCEPTION	Any SQL error
SQLWARNING	SQL warning
Error Code	Specific error (e.g., 1062)

Example:

```
+-------+-------------+------+-----+
| Field | Type        | Null | Key |
+-------+-------------+------+-----+
| sid   | int         | NO   | PRI |
| sname | varchar(50) | YES  |     |
```

```
CREATE PROCEDURE insert_student(
  IN p_id INT,
  IN p_name VARCHAR(50)
)
BEGIN
```

```
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    SELECT 'Error: Duplicate ID or SQL Error' AS Message;
  END;

  INSERT INTO student VALUES (p_id, p_name);
  SELECT 'Record Inserted Successfully' AS Message;
END$$

DELIMITER ;
```

Output:

CALL insert_student(1, Faahim);

```
mysql> CALL insert_student(1, 'Faahim');
+-----------------------------+
| Message                     |
+-----------------------------+
| Record Inserted Successfully |
+-----------------------------+
1 row in set (0.02 sec)
```

CALL insert_student(1, 'Ram');

```
mysql> CALL insert_student(1, 'Ram');
+---------------------------------+
| Message                         |
+---------------------------------+
| Error: Duplicate ID or SQL Error |
+---------------------------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.02 sec)
```

Practice:

Result

Thus, Procedures, Functions Cursors, Triggers, and Exception Handling in PL/SQL was executed successfully and verified.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 7a **Normalization – 1NF, 2NF, 3NF**
Date:

Aim:

The aim is to introduce and reinforce the concepts of database normalization for applying First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

Concept Explanation:

Normalization is the process of organizing data in a database to:

- Reduce data redundancy
- Eliminate update, insertion, and deletion anomalies
- Improve data integrity and consistency

Normalization is achieved by dividing large tables into smaller, well-structured tables based on rules called Normal Forms**.**

FIRST NORMAL FORM (1NF)
Definition
A table is said to be in First Normal Form (1NF) if:

- **Each field contains atomic (indivisible) values**
- **No multi-valued attributes**
- **No repeating groups**

Example (Not in 1NF)

StudentID	StudentName	Subjects
S101	Ravi	DBMS, OS
S102	Anita	DBMS

Subjects contains multiple values.

Converted to 1NF

StudentID	StudentName	Subject
S101	**Ravi**	**DBMS**
S101	**Ravi**	**OS**
S102	**Anita**	**DBMS**

Practice:

First Normal Form (1NF)

1. Take an unnormalized table)
2. Identify the violation of normalization.
3. Convert the table into First Normal Form (1NF).

SECOND NORMAL FORM (2NF)

Definition

A table is in Second Normal Form (2NF) if:

- It is already in 1NF
- No partial dependency exists (i.e., non-key attributes depend on the entire primary key)

Applies only to tables with composite primary keys.

Example (Not in 2NF)

OrderID	ProductID	CustomerName	Price

Primary Key: (OrderID, ProductID)

- CustomerName depends only on OrderID
- Price depends only on ProductID
- **Partial dependency exists**

Converted to 2NF

ORDER Table

OrderID	CustomerName

PRODUCT Table

ProductID	Price

ORDER_PRODUCT Table

OrderID	ProductID

Practice:

1. Take a table in First Normal Form
2. Identify partial dependency
3. Normalize the table to **2NF**

THIRD NORMAL FORM (3NF)

Definition

A table is in Third Normal Form (3NF) if:

- It is already in 2NF
- No transitive dependency exists (Non-key attribute should not depend on another non-key attribute)

Example (Not in 3NF)

EmpID	EmpName	DeptName	DeptLocation

- DeptLocation depends on DeptName
- DeptName depends on EmpID
- **Transitive dependency exists:** DeptLocation→ DeptName→ EmpID

Converted to 3NF

EMPLOYEE Table

EmpID	EmpName	DeptName

DEPARTMENT Table

DeptName	DeptLocation

Practice:

Tasks:

1. Take a table in Second Normal Form
2. Identify transitive dependency
3. Normalize the table to 3NF

Result

Thus, the concepts of database normalization for applying First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 7b **Normalization – 4NF, 5NF**
Date:

Aim:

The aim is to introduce and reinforce the concepts of database normalization for applying Fourth Normal Form (4NF) and Fifth Normal Form (5NF).

Concept Explanation:

FOURTH NORMAL FORM (4NF)
Definition
A table is in Fourth Normal Form (4NF) if:

- **It is already in Boyce-Codd Normal Form (BCNF)**
- **It has no non-trivial multi-valued dependencies (MVDs)**

Multi-valued dependency (MVD):
When one attribute determines multiple independent values of another attribute.

Example (Not in 4NF)

EmpID	Skill	Project
E1	Java	P1
E1	Java	P2
E1	SQL	P1
E1	SQL	P2

An employee can have multiple skills and multiple projects, independent of each other.

Multi-Valued Dependencies

- **EmpID →→ Skill**
- **EmpID →→ Project**

Converted to 4NF
EMP_SKILL Table

EmpID	Skill

EMP_PROJECT Table

EmpID	Project

Practice:

Tasks:

1. Take a table in 3NF
2. Identify the multi-valued dependencies.
3. Normalize the table to 4NF.

FIFTH NORMAL FORM (5NF)

Definition

A table is in Fifth Normal Form (5NF) if:

- It is already in 4NF
- It has no join dependencies
- The table cannot be further decomposed without losing information

Join Dependency:

When a table can be reconstructed by joining three or more tables, and decomposition is necessary.

Example (Not in 5NF)

Supplier	Part	Project
S1	P1	PR1
S1	P2	PR1
S2	P1	PR2

Relationship exists only when all three attributes are combined

Converted to 5NF

SUPPLIER_PART | Supplier | Part |
SUPPLIER_PROJECT | Supplier | Project |
PART_PROJECT | Part | Project |

Practice:

Tasks:

1. Take a table in 4NF
2. Identify the join dependency.
3. Normalize the table into 5NF.

Result

Thus, the concepts of database normalization for applying Fourth Normal Form (4NF) and Fifth Normal Form (5NF)

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 8 **Transaction and Concurrency Control**
Date:

Aim:

To study and practice transaction and concurrency control in DBMS.

Syntax:

S.No	Command	Purpose	Syntax	Example	Output / Result
1	START TRANSACTION / BEGIN	Starts a new transaction block	START TRANSACTION; or BEGIN;	BEGIN;	Transaction begins; changes are not saved permanently
2	COMMIT	Saves all changes made during the transaction permanently to the database	COMMIT;	COMMIT;	All changes are permanently stored
3	ROLLBACK	Undoes all changes made during the current transaction	ROLLBACK;	ROLLBACK;	Database returns to the previous committed state
4	SAVEPOINT	Sets a point within a transaction to which a rollback can occur	SAVEPOINT savepoint_name;	SAVEPOINT sp1;	Savepoint sp1 is created
5	ROLLBACK TO SAVEPOINT	Rolls back changes up to a specific savepoint	ROLLBACK TO savepoint_name;	ROLLBACK TO sp1;	Changes after savepoint sp1 are undone
6	RELEASE SAVEPOINT	Deletes a savepoint that is no longer needed	RELEASE SAVEPOINT savepoint_name;	RELEASE SAVEPOINT sp1;	Savepoint sp1 is removed

Example:

```
BEGIN;

INSERT INTO account VALUES (101, 'Asha', 5000);

SAVEPOINT sp1;

UPDATE account SET balance = balance - 1000 WHERE acc_no = 101;

ROLLBACK TO sp1;
```

COMMIT;

Sample Output:

```
mysql> select * from account;
+--------+------+---------+
| acc_no | name | balance |
+--------+------+---------+
|    102 | Anu  |    7000 |
+--------+------+---------+
1 row in set (0.03 sec)
```

Practice:

Concurrency Control

Syntax and Definition:

S.No	Type of Lock	Purpose	Example (SQL / Scenario)	Result / Explanation
1	Shared Lock (Read Lock)	Allows multiple transactions to read data but not modify it	LOCK TABLES account READ;	Other users can SELECT, but INSERT/UPDATE/DELETE are blocked
2	Exclusive Lock (Write Lock)	Allows only one transaction to read and write data	LOCK TABLES account WRITE;	No other transaction can read or write the table
3	Row-Level Lock	Locks only the required row, not the whole table	SELECT * FROM account WHERE acc_no=101 FOR UPDATE;	Only row 101 is locked; other rows remain accessible
4	Table-Level Lock	Locks the entire table	LOCK TABLES account WRITE;	Whole table is unavailable to other users
5	Predicate Lock (SERIALIZABLE)	SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; START TRANSACTION; SELECT * FROM account WHERE balance>3000;	INSERT INTO account VALUES (105,'Ravi',9000);	T2 blocked (phantom read prevented)
6	COMMIT	Makes changes of a transaction visible to other transactions	COMMIT;	COMMIT;
7	ROLLBACK	Cancels changes made by a transaction	ROLLBACK;	ROLLBACK;

Example:Table: account

Acc_no	name	balance
101	**Asha**	**3500**
102	**Anu**	**6200**

Example:

S.No	Type of Lock	Transaction-1 (T1) Code	Transaction-2 (T2) Code	Output / Observation
1	Shared Lock (READ)	LOCK TABLES account READ;	SELECT * FROM account; UPDATE account SET balance=5000 WHERE acc_no=101;	SELECT allowed; UPDATE blocked
2	Exclusive Lock (WRITE)	LOCK TABLES account WRITE;	SELECT * FROM account; UPDATE account SET balance=6000 WHERE acc_no=101;	T2 blocked for both read & write
3	Unlocking	UNLOCK TABLES;	SELECT * FROM account;	Lock released; T2 executes
4	Row-Level Lock	START TRANSACTION; SELECT * FROM account WHERE acc_no=101 FOR UPDATE;	UPDATE account SET balance=7000 WHERE acc_no=101;	T2 waits until T1 commits
5	Table-Level Lock	LOCK TABLES account WRITE;	UPDATE account SET balance=8000 WHERE acc_no=102;	Entire table locked
6	Predicate Lock (SERIALIZABLE)	SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; START TRANSACTION; SELECT * FROM account WHERE balance>3000;	INSERT INTO account VALUES (105,'Ravi',9000);	T2 blocked (phantom read prevented)
2	UNLOCK TABLES	UNLOCK TABLES;	SELECT * FROM account;	Lock released; T2 executes successfully

Practice:

Result

Thus, the concepts of transaction and concurrency control was studied and executed.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature

Expt No: 9 **Database Connectivity**
Date:

Aim:

To establish database connectivity between a front-end application and a database, and to perform basic database operations such as inserting and retrieving data.

Steps:

Database connectivity with front end is the process of sending user input from a front-end application to a database, processing it, and displaying the result back to the user.

Steps Involved

1. User enters data in the front-end form
2. Data is sent to the server-side script
3. Server establishes a connection to the database
4. Specify the Database Driver
    1. Load or specify the driver required to communicate with the database.
    2. Example: MySQL driver, Oracle driver, JDBC driver.

        - Python → `mysql-connector`
        - Node.js → `mysql`
        - Java → JDBC

5. Provide Database Connection Details
    1. Specify host name, database name, user name, and password.
6. Establish Connection to the Database
    1. Create a connection object using the database driver.
    2. If connection is successful, communication with database begins.
7. Select the Database
    1. Choose the required database to work with.
    2. Example: USE database_name;
8. Create a Statement / Cursor / Query Object
    1. Used to send SQL commands to the database.
9. Select the Required Table
    1. Choose the table on which operations are to be performed.
    2. Example: SELECT * FROM table_name;
10. Execute SQL Query
    1. Perform database operations such as INSERT, SELECT, UPDATE, DELETE.
11. Process the Result
    1. Retrieve records or check execution status.
12. Commit or Rollback Transaction
    1. Save changes permanently or undo changes.
13. Close the Connection
    1. Release database resources after operation.
14. Result is returned to the server
15. Output is displayed on the front end

Front End: user.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Account Entry</title>
</head>
<body>
   <h2>Account Form</h2>
   <form action="/insert" method="post">
      Account No: <input type="text" name="acc_no"><br><br>
      Name: <input type="text" name="name"><br><br>
      Balance: <input type="text" name="balance"><br><br>
      <input type="submit" value="Submit">
   </form>
</body>
</html>
```

Backend:

Backend program	Execution and output

user_py
from flask import Flask, render_template, request
import mysql.connector
app = Flask(__name__)
# Database connection 	
db = mysql.connector.connect(
  host="localhost",
  user="root",
  password="",
  database="bankdb"
)
cursor = db.cursor()
@app.route('/')
def form():
  return render_template('form.html')
@app.route('/insert', methods=['POST'])
def insert():
  acc_no = request.form['acc_no']
  name = request.form['name']
  balance = request.form['balance']
  sql = "INSERT INTO account VALUES (%s, %s, %s)"
  values = (acc_no, name, balance)
  cursor.execute(sql, values)
  db.commit()
  return "Record inserted successfully"
 	
if __name__ == '__main__':
	Execution:
	D://admin>>python user_reg.py
	
	open browser:
	http://localhost:3000
	
	output if front end
	Record inserted successfully
	
	Database output:
	mysql> SELECT * FROM account;
	
	acc_no | **name** | **balance**
	101 | Ravi | 5000

app.run(debug=True)

user_reg.js

```
const express = require('express');
const mysql = require('mysql');
const bodyParser = require('body-parser');

const app = express();
app.use(bodyParser.urlencoded({ extended: true }));

// Database connection
const db = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: "",
    database: "bankdb"
});

// Connect to database
db.connect((err) => {
    if (err) {
        console.log("Database connection failed");
    } else {
        console.log("Database connected");
    }
});

// Display form
app.get('/', (req, res) => {
    res.sendFile(__dirname + "/form.html");
});
// Insert data
app.post('/insert', (req, res) => {
    const acc_no = req.body.acc_no;
    const name = req.body.name;
    const balance = req.body.balance;

    const sql = "INSERT INTO account VALUES (?, ?, ?)";
    db.query(sql, [acc_no, name, balance], (err, result) => {
        if (err) throw err;
        res.send("Record inserted successfully");
    });
});

app.listen(3000, () => {
    console.log("Server running on port 3000");
```

Execution:

D://admin>>node user_reg.js

open browser:
http://localhost:3000

output if front end
Record inserted successfully

Database output:
mysql> SELECT * FROM account;

acc_no	name	balance
101	Ravi	5000

```
});
```

user_reg.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class InsertServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request,
HttpServletResponse response)
         throws ServletException, IOException {

      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      String acc_no = request.getParameter("acc_no");
      String name = request.getParameter("name");
      String balance = request.getParameter("balance");

      try {
         // Load JDBC Driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // Establish Connection
         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/bankdb",
            "root",
            ""
         );
         // Insert Query
         String sql = "INSERT INTO account VALUES (?, ?, ?)";
         PreparedStatement ps = con.prepareStatement(sql);
         ps.setInt(1, Integer.parseInt(acc_no));
         ps.setString(2, name);
         ps.setInt(3, Integer.parseInt(balance));
         ps.executeUpdate();
         out.println("<h3>Record inserted successfully</h3>");
         con.close();

      } catch (Exception e) {
         out.println(e);
      }
   }
}
```

Execution:

```
D://admin>>javac user_reg.java

open browser:
http://localhost:3000

output if front end
Record inserted successfully

Database output:
mysql> SELECT * FROM
account;

acc_no name   balance
101    Ravi   5000
```

Practice:

Result

Thus, Database connectivity was successfully established and data is moved to and from the frontend and database.

Command Execution (3)	Viva(2)	Total (5)	Faculty Comments and Signature